HAI TIEN

LO5

6. Bài tập tính toán

Bài 1: RNN

$x_t = [1, 0]^T$, $h_{t-1} = [0.5, 0.2]^T$, $b_h = [0, 0]^T$

$W_{xh} = [[0.5, 0.2], [0.1, 0.4]]$ $W_{hh} = [[0.3, 0.1], [0.2, 0.5]]$

1) $W_{xh} \cdot x_t = [0.5 \times 1 + 0.2 \times 0, 0.1 \times 1 + 0.4 \times 0]$
$= [0.5, 0.1]^T$

2) $W_{hh} \cdot h_{t-1} = [0.3 \times 0.5 + 0.1 \times 0.2, 0.2 \times 0.5 + 0.5 \times 0.2]$
$= [0.17, 0.2]^T$

3) $a_t = [0.5, 0.1]^T + [0.17, 0.1]^T + b_h$
$= [0.67, 0.3]^T$

4) $h_t = \tanh(a_t) = \tanh([0.67, 0.3]^T)$
$\approx [0.585, 0.291]^T$

7. Bài 2 - Information flow

Câu "I did not like this movie"

$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b)$

Bước	Token	h_t chứa thông tin
h_1	I	I
h_2	did	I did
h_3	not	I did not (có tín hiệu phủ định)
h_4	like	like (đọc đọc trong ngữ cảnh đã có not)
h_5	this	not like this
h_6	movie	tóm tắt cả câu, dùng để phân loại

Khi đọc "like", "not" đã nằm trong h_3, h_4 đi vào h_4

- qua W_{hh} h_5 và kết hợp (qua tanh) với embedding của "like", nên h_6 có thể mã hoá "không thích". Thông tin này tiếp tục qua h_7, h_8 nhưng bị pha loãng dần.
- So với Average Word2Vec:

$$\text{vector tổng bình} = \frac{v_I + v_did + v_like + v_this + v_movie}{6}$$

"not" chỉ chiếm $\frac{1}{6}$ của tổng, không tương tác với "like" và không có thứ tự.

8. Bài 3 - Sequence order

S_1 = "dog bites man" ; S_2 = "man bites dog"

→ cùng vocabulary

Average Word2Vect: Không phân biệt được, cùng tập từ nên giống hệt ở cả 2 câu (phép cộng giao hoán)

RNN: phân biệt được vì h_t tính tuần tự và phép tính không giao hoán:

~~$S_1: h_3 = f(man, f(dog, h_0))$~~

$S_1: h_3 = f(man, f(bites, f(dog, h_0)))$

$S_2: h_3 = f(dog, f(bites, f(man, h_0)))$

Mỗi từ được học cùng một h_{t-1} khác nhau nên $\tanh(U.x + U.h)$ không đổi cho được, nên h_3 khác nhau "ai cắn ai".

9. Vanishing gradient.

Khi learn + truyền ngược qua thời gian, gradient từ bước T về bước k là tích của nhiều Jacobian

$$\frac{\partial h_t}{\partial h_{t-1}} = diag(1 - h_t^2) . W_{hh}$$

$\frac{\partial h_T}{\partial h_k}$ = tích của $(T-k)$ Ja cobian với $t = \overline{k+1, T}$

- Vì $\tan k' \leq 1$, nếu w_{hh} nhỏ (độ lớn < 1) thì mỗi thừa số < 1 và tích giảm theo hàm mũ. Ví dụ 0.5 mỗi bước, sau 50 bước $0.5^{50} \approx 10^{-15} \approx 0$.
- Tín hiệu lỗi từ cuối chuỗi gần như không tới được token xa, nếu tham số không được cập nhật theo thời xa. Vì vậy RNN khó học dependency dài, và chuỗi càng dài càng nặng.
- Ngược lại, thừa số > 1 làm gradient tăng theo hàm mũ (exploding gradient), thì xử lý bằng gradient clipping.

17. Bài tập tính toán - LSTM gates

$f_t = 0.9$, $i_t = 0.2$, $\tilde{C}_t = 0.5$, $C_{t-1} = 0.8$

$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t = 0.9 \times 0.8 + 0.2 \times 0.5$

$= 0.72 + 0.10 = 0.82$.

Forget gate gần 1: giữ thông tin cũ ($C_t \approx C_{t-1}$)

0: loại bỏ (C_t do $i_t, \tilde{C}_t$ quyết định)

Gate và "van" điều khiển dòng thông tin, C_t được cập nhật bằng phép cộng nên gradient dọc cell state ít bị co hơn nên LSTM giữ dependency dài tốt hơn RNN.